

Date	07 July 2026
Team ID	NA
Project Name	HouseHunt (MERN House Rental Platform)
Phase	Project Documentation

Project Documentation: FSD Documentation Format

1. Introduction & Development Team

- Project Title: HouseHunt – A premium MERN Stack House Rental Platform.

Team Members:

- Hemasundararao Meesala (Lead MERN Developer, Security Architect, & UI/UX Designer)

Dona Sri Divya Nerusu (Team Lead)

- Rajani Meesala (Member)

Harshitha Nulu (Member)

- Varshitha Nagadasi (Member)

2. Project Overview

- HouseHunt is designed to simplify home rental listings and lease management in India's major metropolitan areas. It implements a secure role progression flow enabling Tenants to list properties for admin approval, automatically upgrading their account to an Owner role upon validation.

3. API Interface Specifications

HTTP Verb	Route Endpoint	Role Required	Request Body / Payload	Success Status	Description
POST	/api/users/send-otp	Public	{ "email": "user@gmail.com"} }	200 OK	Generates OTP and emails it.
POST	/api/users/register	Public	{ "name": "John", "email": "user@gmail.com", "phone": "9876", "password": "123", "role": "Tenant", "verificationCode": "123"} }	201 Created	Registers profile.

•	POST	• /api/users/login	• Public	• { "email": "user@gmail.com", "password": "123" }	• 200 OK	• Authenticates and returns JWT.
•	GET	• /api/users/properties	• Public	• Query : city, propertyType, rentMax, bedrooms	• 200 OK	• Fetches active available listings.
•	POST	• /api/users/properties/submit	• Protected (Tenant)	• Form Data (Listing specs + photo files)	• 201 Created	• Creates listing in 'Pending' status.
•	GET	• /api/users/properties/submissions	• Protected (Tenant, Owner)	• None	• 200 OK	• Fetches submissions history.
•	POST	• /api/users/bookings	• Protected (Tenant, Owner)	• { "propertyId": "id", "startDate": "date", "endDate": "date" }	• 201 Created	• Creates booking request. Self-booking is blocked.
•	GET	• /api/users/bookings	• Protected (Tenant, Owner)	• None	• 200 OK	• Retrieves bookings placed.
•	GET	• /api/owners/properties	• Protected (Owner)	• None	• 200 OK	• Retrieves owner's properties.
•	PUT	• /api/owners/bookings/status	• Protected (Owner)	• { "bookingId": "id", "status": "Approved" }	• 200 OK	• Approves/rejects booking request.
•	GET	• /api/admins/properties/pending	• Protected (Admin)	• None	• 200 OK	• Fetches pending property submissions.
•	PUT	• /api/admins/properties/:id/approve	• Protected (Admin)	• Param: id	• 200 OK	• Approves listing and upgrades tenant to Owner.
•	DELETE	• /api/admins/properties/:id	• Protected (Admin)	• Param: id	• 200 OK	• Deletes listing and booking logs.
•	DELETE	• /api/admins/users/:id	• Protected (Admin)	• Param: id	• 200 OK	• Deletes user profile.

-
-

• 4. Database Collections Structure

• A. Users Schema (users)

Field Name	Type	Key Type	Nullable	Description
_id	ObjectId	Primary Key	No	Unique user system ID.
name	String	-	No	Full name of the user.
email	String	Unique	No	Email address.
phone	String	-	No	Contact number.
password	String	-	No	Hashed password.
role	String	Enum	No	Role mapping: Tenant, Owner, Admin.
isVerified	Boolean	-	No	OTP verification flag.
profileImage	String	-	Yes	Relative file path to avatar.

-
-

• B. Properties Schema (properties)

Field Name	Type	Key Type	Nullable	Description
_id	ObjectId	Primary Key	No	Unique property system ID.
ownerId	ObjectId	Foreign Key	No	References the Owner's users._id.
title	String	-	No	Marketing title of the listing.
description	String	-	No	Text description of the house.
rentAmount	Number	-	No	Monthly rental cost.
city	String	-	No	Target city location.

• address	• String	• -	• No	• Street location address.
• bedrooms	• Number	• -	• No	• BHK count.
• bathrooms	• Number	• -	• No	• Bath count.
• images	• Array	• -	• No	• Relative paths of uploaded house photos.
• status	• String	• Enum	• No	• Review status: Pending, Available, Booked, Inactive.

-
-

• 5. Work Breakdown Structure (WBS)

• WBS Code	• Phase / Task Name	• Assigned Role	• Estimated Duration	• Milestone Deliverable
• 1.0	• Requirements & Design Phase	• System Architect & UI Designer	• 5 Days	• Functional Specifications & UI Prototypes
• 1.1	• Compile layout specs & assets specs	• Hemasundar Rao Meesala	• 1 Day	• Style guide sheet completed
• 1.2	• Design UI wireframes & routing	• Dona Sri Divya Nerusu	• 2 Days	• UI wireframes approved
• 1.3	• Map database Mongoose schemas	• Hemasundar Rao Meesala	• 2 Days	• Mongoose models drafted
• 2.0	• Backend Core Development	• Senior Backend Developer	• 10 Days	• Complete running backend REST API
• 2.1	• Initialize Express.js app & router	• Harshitha Nulu	• 2 Days	• App repository initialized
• 2.2	• Establish MongoDB Mongoose connection	• Rajani Meesala	• 1 Day	• DB link confirmed
• 2.3	• Implement auth, hashing, and OTP	• Harshitha Nulu	• 4 Days	• Sign up and login flows complete
• 2.4	• Configure Multer uploads middleware	• Varshitha Nagadasi	• 3 Days	• File upload pipeline tested

• 3.0	• Frontend Client Development	• Senior Frontend Developer	• 10 Days	• Interactive responsive React client
• 3.1	• Bootstrap Vite React project structure	• Varshitha Nagadasi	• 1 Day	• Vite React app initialized
• 3.2	• Configure themes variables	• Hemasun dararao Meesala	• 2 Days	• index.css styles complete
• 3.3	• Code public auth pages (Login, Register)	• Rajani Meesala	• 4 Days	• Verification modals complete
• 3.4	• Build search filter bar & listings cards	• Dona Sri Divya Nerusu	• 3 Days	• Properties grid feed complete
• 4.0	• Role Progression & Moderation	• MERN Developer	• 8 Days	• End-to-end Tenant-to-Owner workflow
• 4.1	• Write property submit router (Pending status)	• Harshitha Nulu	• 2 Days	• Submissions POST route tested
• 4.2	• Code Admin approval controller	• Rajani Meesala	• 2 Days	• Tenant-to-Owner role upgrade complete
• 4.3	• Build renter submit tab & landlord tables	• Dona Sri Divya Nerusu	• 4 Days	• Dashboards UI integrated
• 5.0	• Verification & Final Launch	• QA Engineer & Lead Developer	• 5 Days	• Fully tested production package
• 5.1	• Perform API tests & database checks	• Varshitha Nagadasi	• 2 Days	• API testing logs compiled
• 5.2	• Compile production build (Vite build)	• Hemasun dararao Meesala	• 1 Day	• Static assets folder compiled
• 5.3	• Generate complete documentation guide	• Hemasun dararao Meesala	• 2 Days	• DOCUMENTATION.md generated

-
-

• 6. Risk Management Plan Matrix

• ID	• Risk Description	• Severity	• Probability	• Mitigation Strategy	• Contingency Action Owner
------	--------------------	------------	---------------	-----------------------	----------------------------

• R1	• Windows filesystem locking temporary Vite cache directories (EPERM error).	• 2 (Low)	• Medium	• Configure npm run dev to automatically delete .vite folder in node_modules prior to calling vite.	• Hema sundararao Meesala (Lead Developer)
• R2	• Local MongoDB server connection failure during launch.	• 5 (Critical)	• Low	• Ensure backend connects to local port 27017 with robust catch logs. Pre-program mock database objects inside dashboards as a fallback.	• Rajan i Meesala (DB Architect)
• R3	• Google Mail SMTP authentication credentials blocked or missing in production.	• 4 (High)	• Medium	• Program an automatic Terminal email simulator fallback. If credentials are empty, OTP code prints to console for verification testing.	• Harsh itha Nulu (Security Engineer)
• R4	• Self-booking loop where landlord books their own property, locking active leases.	• 3 (Medium)	• Low	• Implement validation checks in the backend controller matching property.ownerId with req.user.id and block with 400 Bad Request.	• Dona Sri Divya Nerusu (Team Lead)
• R5	• Performance degradation due to unoptimized property image loading size.	• 3 (Medium)	• Medium	• Constraint Multer files upload to standard size limits (max 5MB per file) and compress/optimize image	• Varsh itha Nagadasi (Frontend Member)

				rendering grids in React.	
--	--	--	--	---------------------------------	--

-
-
-